Pravartak

Pravartak is a static, markdown-first learning platform for AI, cloud, and DevOps. It uses Astro and Starlight for documentation, Marp for slides and PDF output, and a small browser-native annotation layer.

It builds to plain files in `dist/` and needs no backend, database, account system, or runtime CDN. The same output works on GitHub Pages, Cloudflare Pages, and any static host.

Quick start

Requirements: Node.js 22.12 or newer (the current Node 22 LTS line is used in CI).

```
npm install
npm run dev
```

Useful commands:

```
npm run check     # Astro and TypeScript checks
npm run build     # verified production build in dist/
npm run preview   # serve the production build locally
npm run slides:pdf # refresh PDFs after editing slides
```

Where authors work

```
src/content/
├── docs/                       # lessons and landing pages
│   ├── llm-foundations/
│   ├── ...
│   └── specialized/
└── slides/                     # standalone slide decks
```

Infrastructure lives elsewhere. A normal content edit should only touch `src/content/docs/` and `src/content/slides/`.

Add a lesson

Create a `.md` file inside the relevant module:

```
---
title: Hybrid search
```

```
description: Combine keyword and vector retrieval.
sidebar:
  order: 2
---

# Hybrid search

Lesson content in normal **Markdown**.

:::tip
Starlight callouts, tables, code fences, and task lists work automatically.
:::
```

The configured sidebar discovers it automatically. Use `.mdx` only when a lesson genuinely needs an imported Astro component.

Add a slide deck

Create `src/content/slides/my-deck.md`:

```
---
marp: true
title: My deck
description: A one-line summary
theme: default
size: 16:9
paginate: true
---

# First slide

Opening idea

<!-- notes
Private notes for this slide.
-->

---

## Second slide

- One idea
- One example
- One question
```

A line containing only `---` separates slides. The build creates `/slides/my-deck/`. Link to it from a lesson with a relative link so GitHub project Pages and root-hosted sites both work:

```
[Open slides](../../slides/my-deck/)
```

Keep slides concise. Markdown headings, lists, tables, blockquotes, and fenced code blocks are supported. Deck source is compiled at build time; no browser markdown parser or CDN package is required.

After changing a deck, run `npm run slides:pdf` before the site build. This refreshes the static PDF download in `public/slides/` using Marp CLI.

Annotation behavior

The slide viewer supports a laser pointer, pen, highlighter, eraser, three sizes, colors, undo/redo, clearing, keyboard shortcuts, touch swipes, notes, true presentation fullscreen, and Marp PDF downloads.

- Strokes use normalized coordinates, so they stay aligned after resize/fullscreen.
- Work autosaves to `localStorage` per deck and browser.
- **Export** downloads a portable JSON backup.
- **Import** validates size and data shape before restoring strokes.
- No annotation data is sent anywhere.

This is deliberately local-only. Collaborative annotations would require a backend and are outside the static-site boundary.

Architecture decisions

The original plan was directionally good, but three choices worked against reliability and maintainability:

1. **Runtime Marp from a CDN** made slides depend on third-party JavaScript and network access. Pravartak uses Marp Core locally during the Astro build and Marp CLI for PDF output.
2. **Build-time Kroki requests** made every deployment depend on an external API. The initial platform avoids network-dependent diagrams; diagrams can be committed as SVG assets or a local build plugin can be added later.
3. **Two slide components with tightly coupled canvases** added lifecycle complexity. One static slide route owns navigation, notes, drawing, storage, import, and export with browser-native JavaScript.

Astro/Starlight remains a strong fit because content routing, sidebar navigation, search, syntax highlighting, table of contents, dark/light themes, and accessibility are maintained upstream. Custom code is limited to the slide route and design tokens.

Deploy to Cloudflare Pages

Connect the Git repository in **Workers & Pages** → **Create application** → **Pages** and use:

Setting	Value
Build command	`npm run build`
Build output directory	`dist`
Node version	`22`

No Cloudflare adapter or Pages Function is needed because the output is fully static. Root-hosted Cloudflare builds use the default `BASE_PATH=/`.

For a direct upload after building:

```
npx wrangler pages deploy dist --project-name pravartak
```

`public/_headers` adds conservative security headers and long-lived caching for hashed Astro assets.

Deploy to GitHub Pages

The workflow at `.github/workflows/deploy-pages.yml` builds and deploys pushes to `main`. In the repository settings, set **Pages** → **Source** to **GitHub Actions**.

The workflow calculates the path automatically:

- `https://owner.github.io/repository/` uses `BASE_PATH=/repository`.
- `https://owner.github.io/` from an `owner.github.io` repository uses `BASE_PATH=/`.

For a manual project-site build:

```
SITE_URL=https://OWNER.github.io BASE_PATH=/REPOSITORY npm run build
```

Quality checklist

Before merging content or code:

1. Run `npm run build`.
2. Open the production preview and check desktop plus a narrow mobile viewport.

3. Follow lesson and slide links from a nested page.
4. Navigate a deck with buttons, keyboard, and touch.
5. Draw on multiple slides; resize; refresh; export; clear; import.
6. Check browser console errors and keyboard focus visibility.

The complete starter lesson is **AWS Deployment**. Other sections contain content outlines ready for future markdown additions.